Universidad Autónoma de Sinaloa

UNIDAD ACADEMICA FACULTAD DE INGENIERIA MOCHIS

Licenciatura en Ingeniería de Software

Alumnos:
Ortiz Leal Erik

Maestro:
Dr. Gibran Uriel Lopez Coronel

Materia:
Desarrollo Móvil Nativo

Actividad:
Actividad 7.1. Práctica 08 - SQLite / Room Database

11/10/2026

1. Introducción y Objetivos de la Práctica

El propósito de esta práctica es implementar persistencia de datos local en Android utilizando Room, la biblioteca de Jetpack que actúa como una capa de abstracción sobre SQLite. Room reduce el código repetitivo, valida las consultas SQL en tiempo de compilación y se integra de forma nativa con corrutinas y Flow.

Además, se integra Room de manera reactiva con Jetpack Compose: el DAO devuelve un Flow y la interfaz lo consume con collectAsState(), de modo que la lista de usuarios se redibuja automáticamente cuando los datos cambian. En el enfoque clásico con SQLiteOpenHelper era necesario escribir sentencias SQL a mano, manejar cursores y avisar manualmente a la vista de cada cambio; con Room basta con declarar una entidad, un DAO y la base de datos.

2. Desarrollo Técnico Paso a Paso

Paso 1 y 2: Creación del Proyecto

Se inició un nuevo proyecto en Android Studio con la plantilla 'Empty Activity', nombrado Practica8 bajo el paquete com.uas.practica8, utilizando Kotlin, Jetpack Compose y Material Design 3.

Paso 3: Arquitectura de Paquetes

Para mantener una arquitectura ordenada se crearon tres paquetes dentro de com.uas.practica8:

- **data:** contiene la entidad Usuario.kt y el DAO UsuarioDao.kt.
- **domain:** contiene AppDatabase.kt, el punto de acceso a la base de datos.
- **Presentation:** contiene MainScreen.kt, la interfaz en Compose.

Paso 4: Plugin KSP (build.gradle.kts del Proyecto)

Room genera código en tiempo de compilación mediante KSP (Kotlin Symbol Processing), por lo que primero se declaró el plugin a nivel de proyecto. Como este proyecto usa Kotlin 2.2.10 y Android Gradle Plugin 9.3.3, se utilizó la versión 2.3.8 de KSP; el formato de versión del manual (1.9.24-1.0.20) corresponde a Kotlin 1.9 y no es compatible.

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.compose) apply false
    id("com.google.devtools.ksp") version "2.3.8" apply false
}
```

Paso 5: Dependencias de Room (build.gradle.kts del Módulo app)

En el módulo app se aplicó el plugin de KSP (sin versión, porque ya se declaró en el proyecto) y se agregaron las tres dependencias de Room. Se usó la versión 2.8.4, más reciente que la 2.6.1 del manual. Después de modificar los archivos se ejecutó Sync Now.

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    id("com.google.devtools.ksp")
}

dependencies {
    // ... dependencias de Compose
    implementation("androidx.room:room-runtime:2.8.4")
    implementation("androidx.room:room-ktx:2.8.4")
    ksp("androidx.room:room-compiler:2.8.4")
}
```

- **room-runtime:** núcleo de Room.
- **room-ktx:** agrega soporte de corrutinas (suspend) y Flow.
- **room-compiler (ksp):** genera en compilación la implementación de la base de datos y del DAO.

Paso 6: Entidad (data/Usuario.kt)

La entidad define la tabla. @Entity indica que la clase es una tabla llamada 'usuarios', y @PrimaryKey(autoGenerate = true) hace que Room asigne el id automáticamente (1, 2, 3...), por lo que al insertar solo se proporciona el nombre.

```
package com.uas.practica8.data

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "usuarios")
data class Usuario(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String
)
```

Paso 7: DAO (data/UsuarioDao.kt)

El DAO (Data Access Object) define los métodos para acceder a la base de datos. Al devolver un Flow, Compose sabe cuándo actualizar la interfaz automáticamente. Las operaciones de escritura usan suspend para ejecutarse en segundo plano con corrutinas.

```
package com.uas.practica8.data

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query
import kotlinx.coroutines.flow.Flow

@Dao
interface UsuarioDao {
    @Query("SELECT * FROM usuarios")
    fun obtenerTodos(): Flow<List<Usuario>>

    @Insert
    suspend fun insertar(usuario: Usuario)
}
```

Paso 8: Base de Datos (domain/AppDatabase.kt)

AppDatabase es el punto de acceso principal a la conexión con la base de datos. Se implementó con el patrón Singleton: @Volatile garantiza que todos los hilos vean el valor

actual de INSTANCE, y synchronized(this) asegura que si dos partes de la app piden la base al mismo tiempo, solo una la cree. Así se evita abrir múltiples conexiones.

```
package com.uas.practica8.domain

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
import com.uas.practica8.data.Usuario
import com.uas.practica8.data.UsuarioDao

@Database(entities = [Usuario::class], version = 1)
abstract class AppDatabase : RoomDatabase() {

    abstract fun usuarioDao(): UsuarioDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "mi_base_datos"
                ).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

Paso 9: Pantalla Principal (Presentation/MainScreen.kt)

En MainScreen se integró la base de datos con una lista dinámica, combinando LazyColumn con la lectura reactiva de Room mediante collectAsState(). Un OutlinedTextField captura el nombre y el botón lo guarda con dao.insertar(...) dentro de coroutineScope.launch(Dispatchers.IO), para no bloquear el hilo principal. La lista se actualiza sola porque observa el Flow del DAO.

```
package com.uas.practica8.Presentation

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp
import com.uas.practica8.data.Usuario
import com.uas.practica8.domain.AppDatabase
import kotlinx.coroutines.Dispatchers
```

```
import kotlinx.coroutines.launch

@Composable
fun MainScreen() {
    val context = LocalContext.current
    val db = remember { AppDatabase.getDatabase(context) }
    val dao = db.usuarioDao()

    val coroutineScope = rememberCoroutineScope()

    // collectAsState() convierte el Flow en un State: la UI se redibuja sola
    val usuarios by dao.obtenerTodos().collectAsState(initial = emptyList())

    var nuevoNombre by remember { mutableStateOf("") }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
            .padding(top = 32.dp)
    ) {
        OutlinedTextField(
            value = nuevoNombre,
            onValueChange = { nuevoNombre = it },
            label = { Text("Nombre del usuario") },
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(modifier = Modifier.height(8.dp))

        Button(
            onClick = {
                if (nuevoNombre.isNotBlank()) {
                    val nombre = nuevoNombre
                    coroutineScope.launch(Dispatchers.IO) {
                        dao.insertar(Usuario(nombre = nombre))
                    }
                    nuevoNombre = ""
                }
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Guardar Usuario")
        }

        Spacer(modifier = Modifier.height(16.dp))
        Text("Lista de Usuarios Registrados:", style = MaterialTheme.typography.titleMedium)
        Spacer(modifier = Modifier.height(8.dp))

        LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {
            items(usuarios) { usuario ->
                Card(modifier = Modifier.fillMaxWidth()) {
                    Text(
                        text = "ID: ${usuario.id} | Nombre: ${usuario.nombre}",
                        modifier = Modifier.padding(16.dp)
                    )
                }
            }
        }
```

```
    }
}
```

Paso 10: Entry Point (MainActivity.kt)

En MainActivity se dejó únicamente la llamada a MainScreen() dentro de setContent, junto con enableEdgeToEdge().

```
package com.uas.practica8

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import com.uas.practica8.Presentation.MainScreen

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MainScreen()
        }
    }
}
```

3. Problemas Encontrados y Soluciones

- **Versión de KSP incompatible.** El manual indica KSP 1.9.24-1.0.20, pero el proyecto usa Kotlin 2.2.10 y AGP 9.3.3. Se cambió a KSP 2.3.8 y Room 2.8.4, y la sincronización de Gradle terminó sin errores.
- **La app se cerraba al abrir.** Logcat mostró 'Cannot find implementation for AppDatabase. AppDatabase_Impl does not exist'. La causa fue que faltaba la línea ksp("androidx.room:room-compiler:2.8.4") en las dependencias, por lo que Room no generó su implementación. Se agregó la línea y se eliminó una dependencia room3 que no correspondía a esta práctica.
- **Los nombres se guardaban vacíos.** La corrutina leía nuevoNombre después de que el campo de texto ya se había limpiado, así que se insertaba una cadena vacía. Se resolvió copiando el valor a una variable local (val nombre = nuevoNombre) antes de lanzar la corrutina.

4. Evidencias de Ejecución en Dispositivo Físico

La aplicación fue instalada y probada en un dispositivo físico. En la Figura 1 se muestra la lista de usuarios leída desde la base de datos Room.

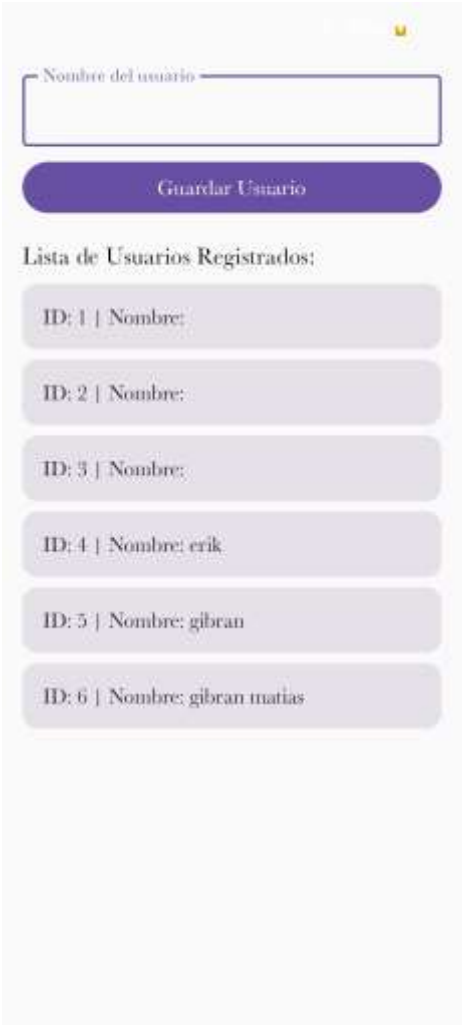


Figura 1: Lista de usuarios registrados con Room

- Los registros tienen un ID autoincremental (1 a 6) asignado por Room.
- Los registros 1 a 3 corresponden a pruebas iniciales hechas antes de corregir el error de captura del nombre; los registros 4 a 6 ('erik', 'gibran' y 'gibran matias') se guardaron con el nombre capturado.
- Los registros de ejecuciones anteriores siguen presentes al volver a abrir la aplicación, lo que confirma la persistencia de los datos.

5. Enlace al Repositorio de GitHub

Repositorio Oficial: El código fuente de la Práctica 08 y de las prácticas anteriores se encuentra versionado en el repositorio del curso:

https://github.com/erik005-col/Pr-cticas-del-curso-Desarrollo-de-Aplicaciones-Nativas

Ruta del proyecto: /Practica8
Rama principal: main
Estado del Build: BUILD SUCCESSFUL

6. Conclusiones

A través de esta práctica se logró implementar persistencia de datos local en Android con Room. Se comprendió cómo una entidad (@Entity), un DAO (@Dao) y una clase de base de datos (@Database) reemplazan el manejo manual de SQL y cursores de SQLite, y cómo KSP genera en compilación el código que los une.

También se comprobó la utilidad del enfoque reactivo: al devolver un Flow desde el DAO y consumirlo con collectAsState(), la interfaz de Compose se actualiza automáticamente cada vez que se inserta un registro, sin código adicional. Las funciones suspend y el uso de Dispatchers.IO permiten escribir en la base de datos sin bloquear el hilo principal, y el patrón Singleton garantiza una única conexión a la base de datos. Por último, la depuración de los errores de configuración (versiones de KSP y Room, y la captura del valor antes de lanzar la corrutina) reforzó la comprensión de cómo se coordinan estas piezas.